SW Pilot 학습보고서 <과정 9>

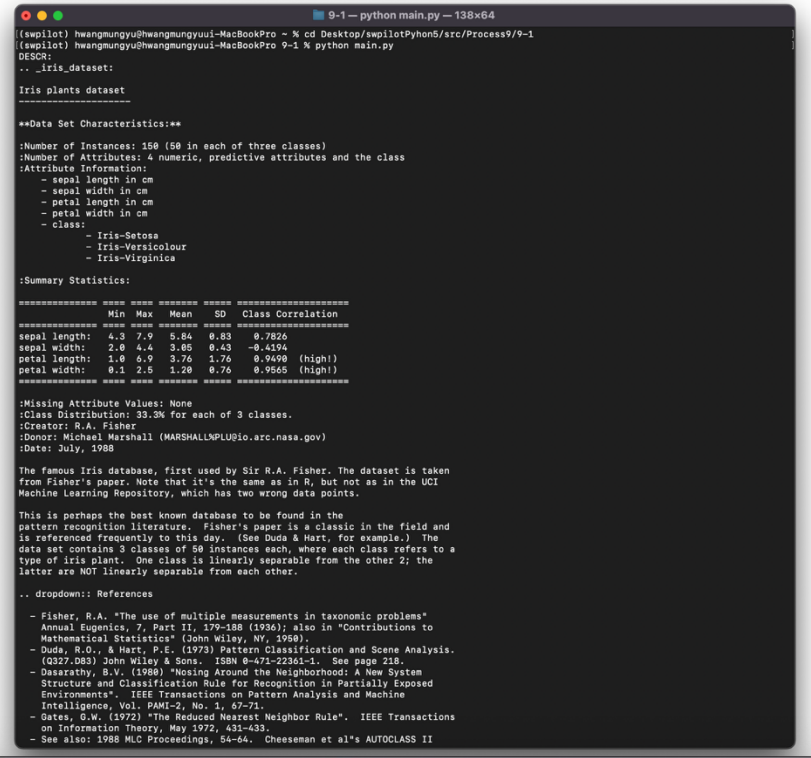
파이썬 5팀 황문규

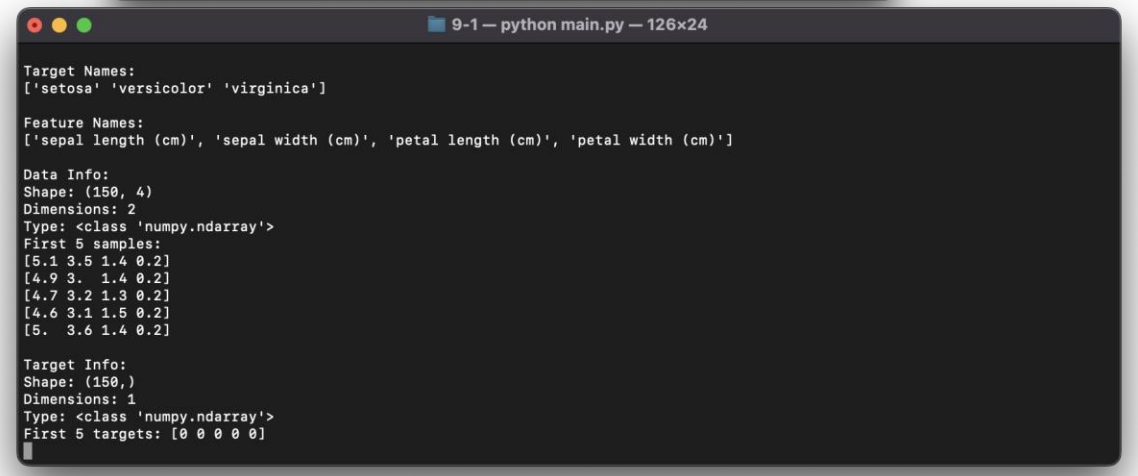
문제 1. 식물 분류 프로젝트 - 코드

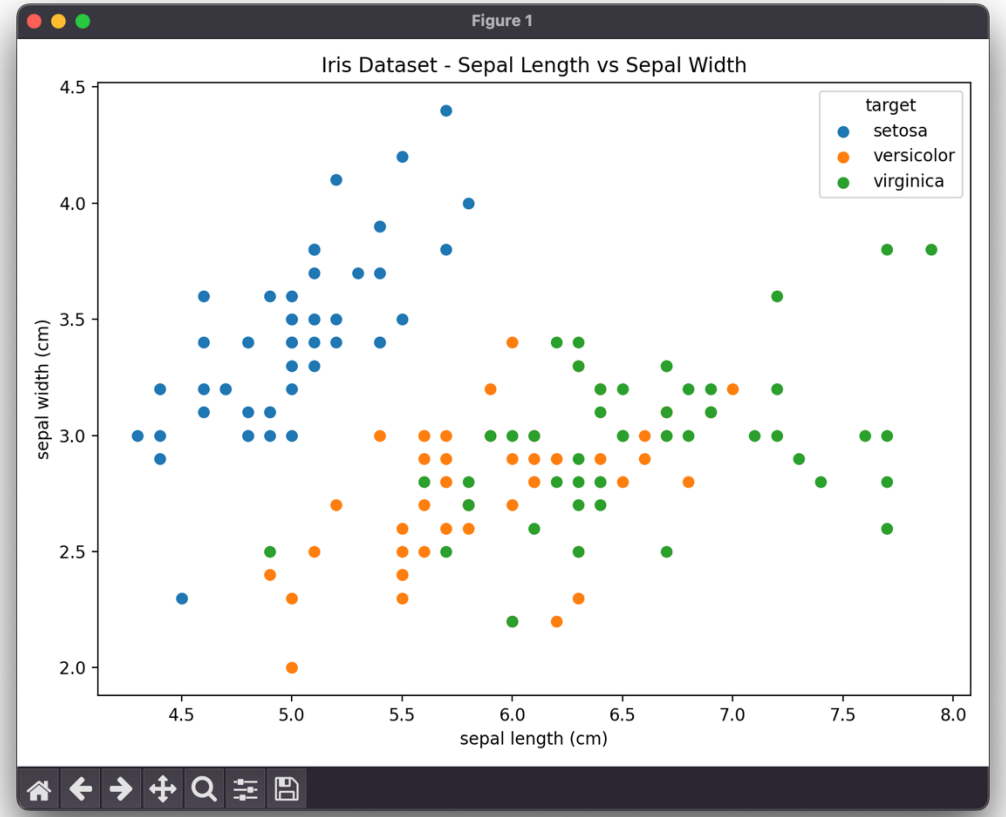
```python
# 과정 9 - (문제1) 식물 분류 프로젝트
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# [보너스 과제] 데이터 분포 시각화
def plot_iris_scatter(data, target, target_names):
    plt.figure(figsize=(8, 6))
    for class_idx, class_label in enumerate(target_names):
        xs = data[target == class_idx, 0]  # sepal length
        ys = data[target == class_idx, 1]  # sepal width
        plt.scatter(xs, ys, label=class_label)

    plt.xlabel('sepal length (cm)')
    plt.ylabel('sepal width (cm)')
    plt.title('Iris Dataset - Sepal Length vs Sepal Width')
    plt.legend(title='target')
    plt.tight_layout()
    plt.show()
```

```python
def main():
    iris_dataset = load_iris()

    print('DESCR:')
    print(iris_dataset['DESCR'])

    print('\nTarget Names:')
    print(iris_dataset['target_names'])

    print('\nFeature Names:')
    print(iris_dataset['feature_names'])

    data = iris_dataset['data']
    print('\nData Info:')
    print('Shape:', data.shape)
    print('Dimensions:', data.ndim)
    print('Type:', type(data))
    print('First 5 samples:')
    for sample in data[:5]:
        print(sample)

    target = iris_dataset['target']
    print('\nTarget Info:')
    print('Shape:', target.shape)
    print('Dimensions:', target.ndim)
    print('Type:', type(target))
    print('First 5 targets:', target[:5])

    target_names = iris_dataset['target_names']

    # [보너스 과제] 데이터 분포 시각화
    plot_iris_scatter(data, target, target_names)

if __name__ == '__main__':
    main()
```

문제 1. 식물 분류 프로젝트 - 구현화면





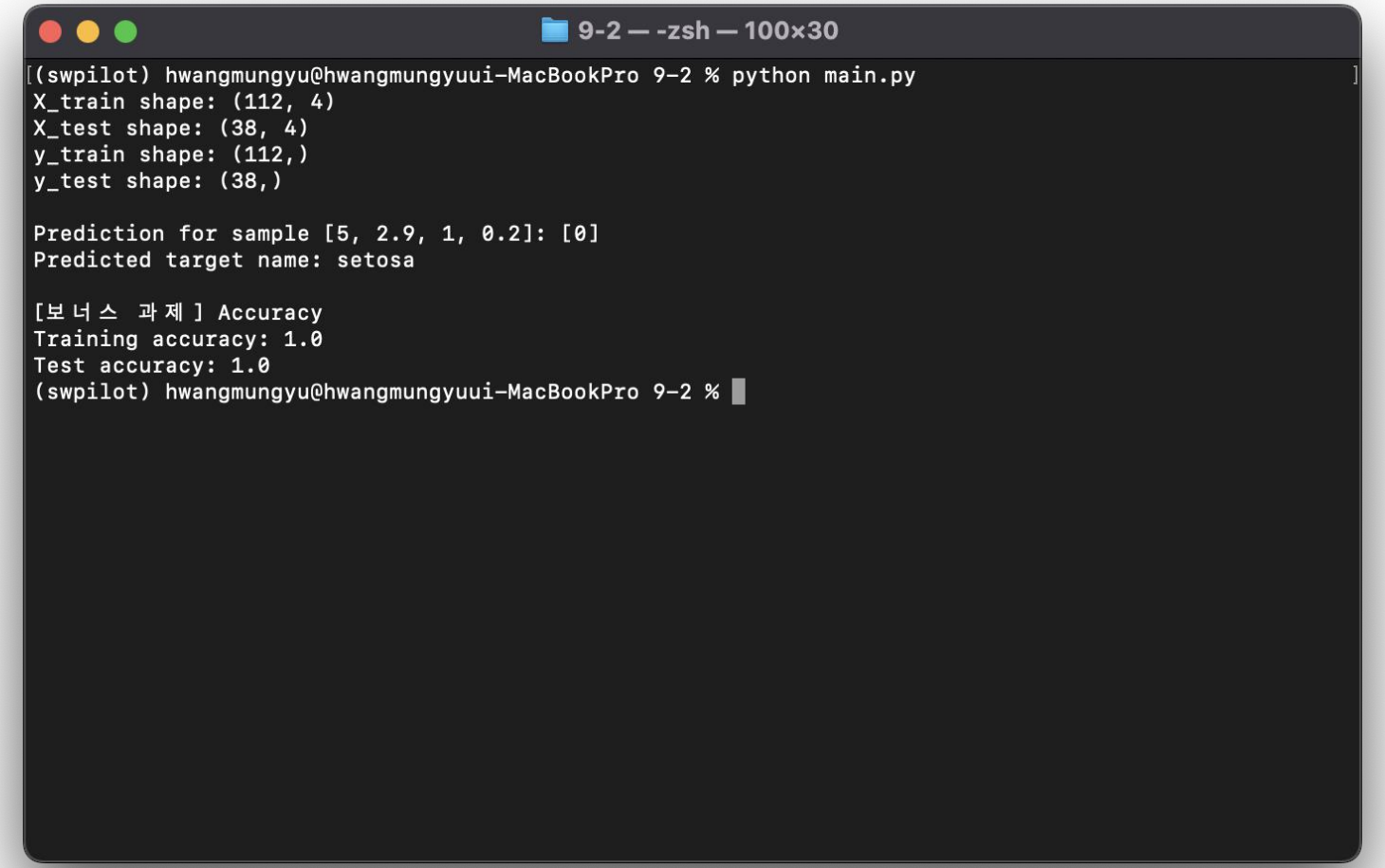


문제 2. 컴퓨터에게 학습을 시켜보자 - 코드

```python
# 과정 9 - (문제2) 컴퓨터에게 학습을 시켜보자
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

def main():
    # iris 데이터셋 로드
    iris = load_iris()
    data = iris['data']
    target = iris['target']

    # train/test 데이터 분할
    x_train, x_test, y_train, y_test = train_test_split(
        data, target, test_size=0.25, random_state=42
    )

    # 분할된 데이터 정보 출력
    print('X_train shape:', x_train.shape)
    print('X_test shape:', x_test.shape)
    print('y_train shape:', y_train.shape)
    print('y_test shape:', y_test.shape)
```

```python
    # 모델 학습 (KNN, 이웃 수 = 1)
    model = KNeighborsClassifier(n_neighbors=1)
    model.fit(x_train, y_train)

    # 새로운 데이터 예측
    sample = [[5, 2.9, 1, 0.2]]
    prediction = model.predict(sample)
    print('\nPrediction for sample [5, 2.9, 1, 0.2]:', prediction)
    print('Predicted target name:', iris['target_names'][prediction[0]])

    # [보너스 과제] 모델 평가
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    print('\n[보너스 과제] Accuracy')
    print('Training accuracy:', train_score)
    print('Test accuracy:', test_score)


if __name__ == '__main__':
    main()
```

문제 2. 컴퓨터에게 학습을 시켜보자 - 구현화면

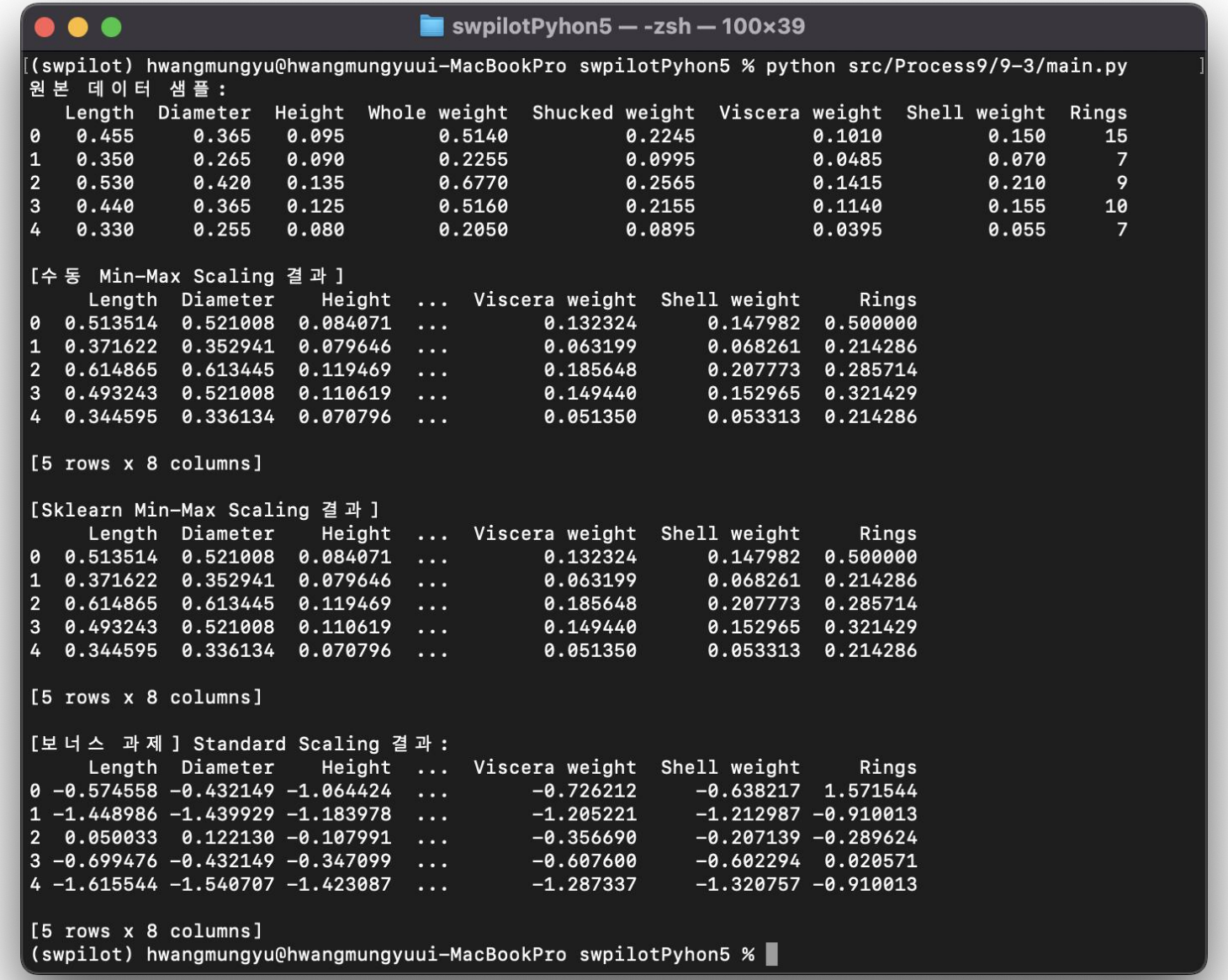


문제 3. 데이터 전처리 Min-Max - 코드

```python
# 과정 9 - (문제3) 데이터 전처리 Min-Max
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler  # [보너스 과제 포함]

def load_abalone_data(data_path, attr_path):
    with open(attr_path, 'r', encoding='utf-8') as f:
        columns = [line.strip() for line in f if line.strip()]

    df = pd.read_csv(data_path, header=None, names=columns)
    return df

def min_max_scaling_manual(df):
    scaled_df = df.copy()
    for column in scaled_df.columns:
        col_min = scaled_df[column].min()
        col_max = scaled_df[column].max()
        scaled_df[column] = (scaled_df[column] - col_min) / (col_max - col_min)
    return scaled_df

def main():
    # 데이터 로드
    data_path = 'src/Process9/9-3/9-3-abalone.txt'
    attr_path = 'src/Process9/9-3/9-3-abalone_attributes.txt'
    df = load_abalone_data(data_path, attr_path)
```

```python
    # label 추출 및 Sex 컬럼 제거
    label = df['Sex']
    df = df.drop(columns=['Sex'])

    print('원본 데이터 샘플:')
    print(df.head())

    # 수동 Min-Max Scaling
    scaled_manual = min_max_scaling_manual(df)
    print('\n[수동 Min-Max Scaling 결과]')
    print(scaled_manual.head())

    # Scikit-learn Min-Max Scaling
    scaler = MinMaxScaler()
    scaled_auto = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
    print('\n[Sklearn Min-Max Scaling 결과]')
    print(scaled_auto.head())

    # [보너스 과제] Standard Scaling
    std_scaler = StandardScaler()
    scaled_standard = pd.DataFrame(std_scaler.fit_transform(df), columns=df.columns)
    print('\n[보너스 과제] Standard Scaling 결과:')
    print(scaled_standard.head())


if __name__ == '__main__':
    main()
```

문제 3. 데이터 전처리 Min-Max - 구현화면

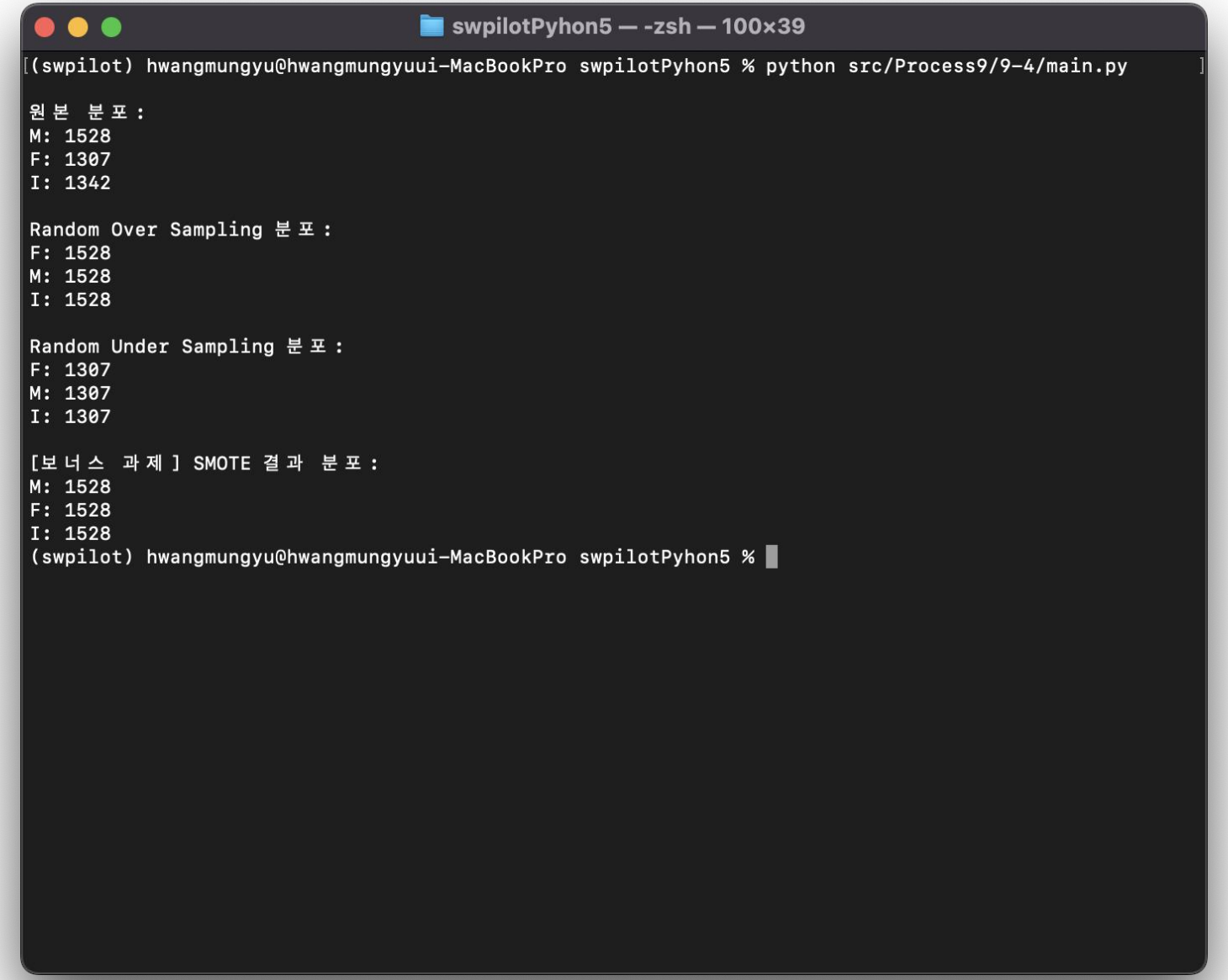


문제 4. 데이터 전처리 Sampling - 코드

```python
# 과정9 - (문제4) 데이터 전처리 Sampling
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
from sklearn.utils import resample
from imblearn.over_sampling import SMOTE  # [보너스 과제]

def load_abalone_data(data_path, attr_path):
    with open(attr_path, 'r', encoding='utf-8') as f:
        columns = [line.strip() for line in f if line.strip()]

    df = pd.read_csv(data_path, header=None, names=columns)
    return df

def random_over_sampling(data, label):
    df = data.copy()
    df['label'] = label
    max_count = df['label'].value_counts().max()
    resampled_list = []

    for value, group in df.groupby('label'):
        if len(group) < max_count:
            upsampled = resample(group, replace=True, n_samples=max_count, random_state=42)
        else:
            upsampled = group
        resampled_list.append(upsampled)

    result = pd.concat(resampled_list).sample(frac=1, random_state=42).reset_index(drop=True)
    return result.drop(columns=['label']), result['label']

def random_under_sampling(data, label):
    df = data.copy()
    df['label'] = label
    min_count = df['label'].value_counts().min()
    resampled_list = []

    for value, group in df.groupby('label'):
        downsampled = resample(group, replace=False, n_samples=min_count, random_state=42)
        resampled_list.append(downsampled)

    result = pd.concat(resampled_list).sample(frac=1, random_state=42).reset_index(drop=True)
    return result.drop(columns=['label']), result['label']

def apply_smote(data, label):
    smote = SMOTE(random_state=42)
    return smote.fit_resample(data, label)
```

```python
def print_label_distribution(label, title):
    print(f'\n{title} 분포:')
    counts = Counter(label)
    for k, v in counts.items():
        print(f'{k}: {v}')

def main():
    data_path = 'src/Process9/9-4/9-4-abalone.txt'
    attr_path = 'src/Process9/9-4/9-4-abalone_attributes.txt'

    # 데이터 로드
    df = load_abalone_data(data_path, attr_path)

    # label: 성별, data: 수치형 정보
    label = df['Sex']
    data = df.drop(columns=['Sex'])

    # label을 숫자로 인코딩 (SMOTE를 위해 필요)
    le = LabelEncoder()
    encoded_label = le.fit_transform(label)

    # 원본 분포
    print_label_distribution(label, '원본')

    # Random Over Sampling
    over_data, over_label = random_over_sampling(data, label)
    print_label_distribution(over_label, 'Random Over Sampling')

    # Random Under Sampling
    under_data, under_label = random_under_sampling(data, label)
    print_label_distribution(under_label, 'Random Under Sampling')

    # [보너스 과제] SMOTE
    smote_data, smote_label = apply_smote(data, encoded_label)
    decoded_smote_label = le.inverse_transform(smote_label)
    print_label_distribution(decoded_smote_label, '[보너스 과제] SMOTE 결과')

if __name__ == '__main__':
    main()
```

문제 4. 데이터 전처리 Sampling - 구현화면

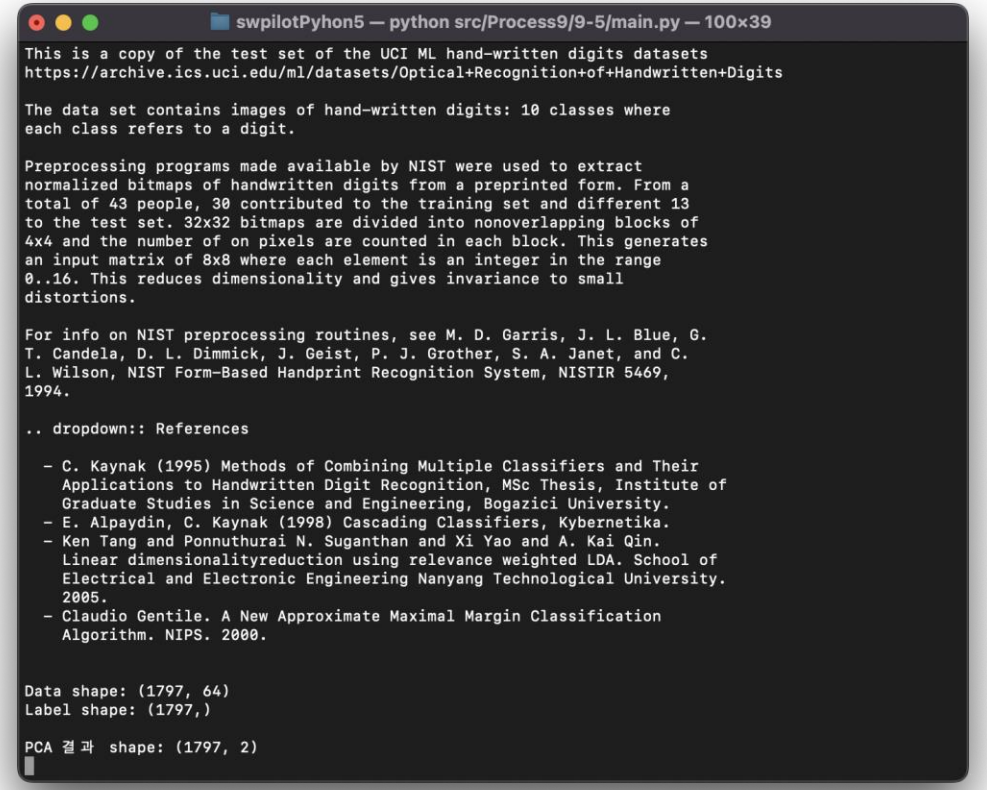


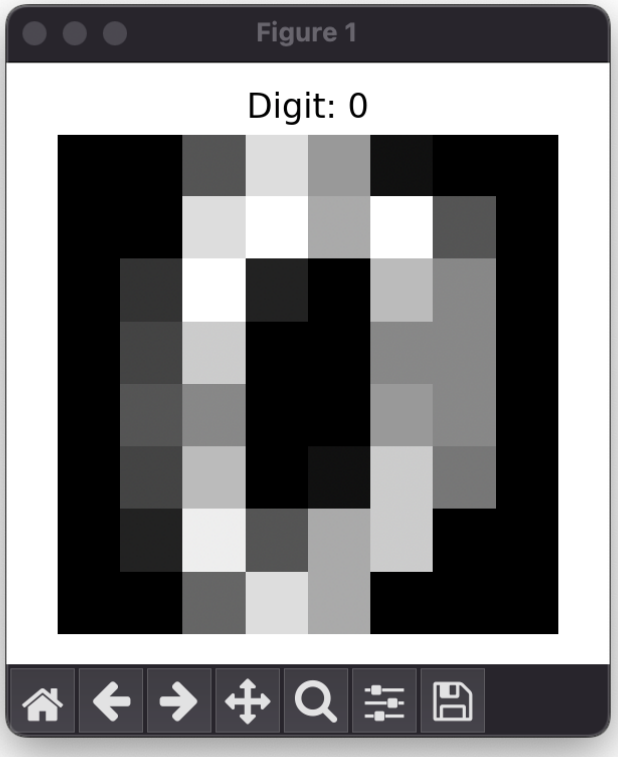
문제 5. 차원의 저주를 풀어라 - 코드

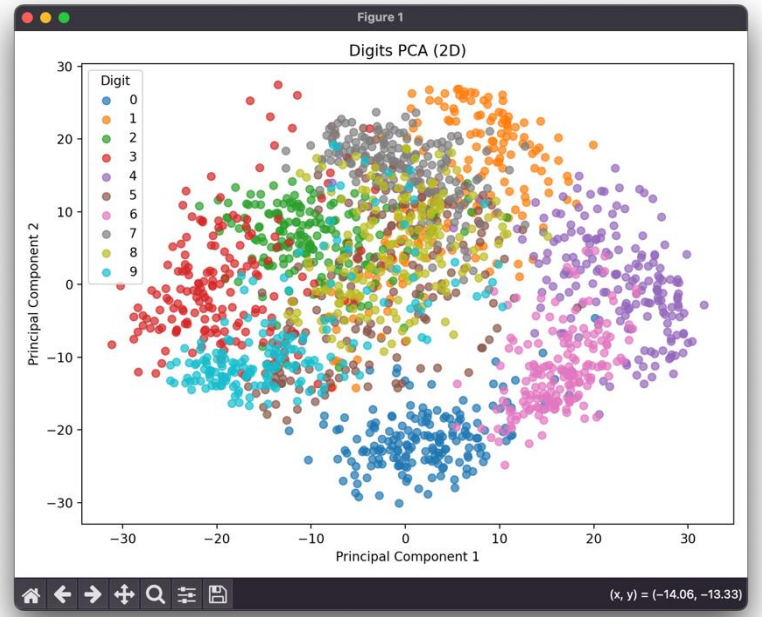
```python
# 과정 9 - (문제5) 차원의 저주를 풀어라
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt  # [보너스 과제]
import numpy as np

def main():
    # 데이터셋 불러오기
    digits = load_digits()

    # DESCR 항목 출력
    print('DESCR:')
    print(digits['DESCR'])

    # data, label 추출
    data = digits['data']
    label = digits['target']

    print('\nData shape:', data.shape)
    print('Label shape:', label.shape)

    # 첫 번째 샘플을 8x8 행렬로 변환하여 시각 확인
    first_image = data[0].reshape((8, 8))

    plt.figure(figsize=(3, 3))
    plt.imshow(first_image, cmap='gray')
    plt.title(f'Digit: {label[0]}')
    plt.axis('off')
    plt.tight_layout()
    plt.show()
```

```python
    # PCA를 통한 2차원 축소
    pca = PCA(n_components=2)
    data_pca = pca.fit_transform(data)

    print('\nPCA 결과 shape:', data_pca.shape)

    # [보너스 과제] 2차원 시각화
    plt.figure(figsize=(8, 6))
    for digit in np.unique(label):
        idx = label == digit
        plt.scatter(data_pca[idx, 0], data_pca[idx, 1], label=str(digit), alpha=0.7)

    plt.title('Digits PCA (2D)')
    plt.xlabel('Principal Component 1')
    plt.ylabel('Principal Component 2')
    plt.legend(title='Digit')
    plt.grid(False)  # 격자 제거
    plt.tight_layout()
    plt.show()


if __name__ == '__main__':
    main()
```

문제 5. 차원의 저주를 풀어라 - 구현화면





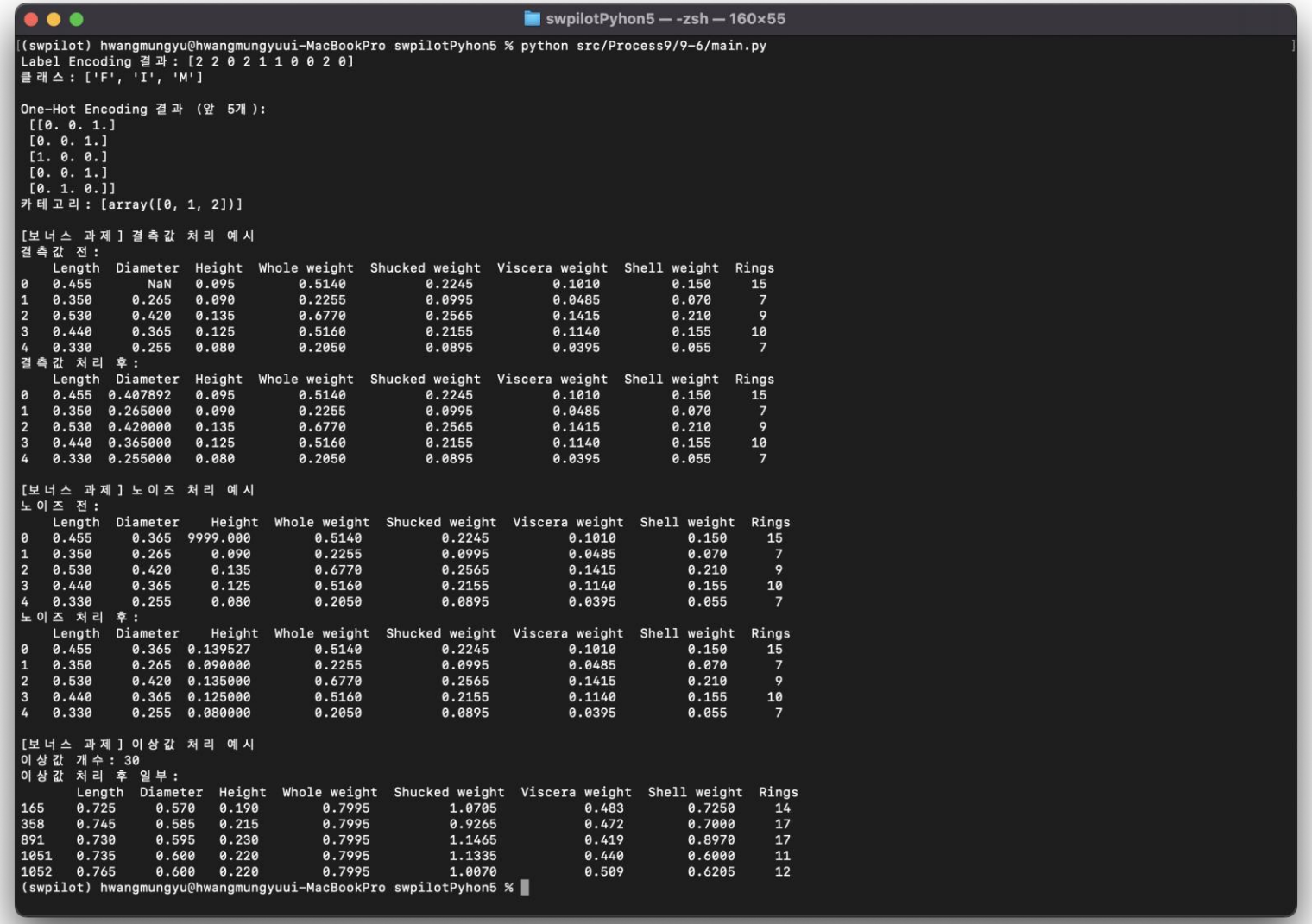


문제 6. 분명하게 말해 두기 - 코드

```python
# 과정 9 - (문제6) 분명하게 말해 두기
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import numpy as np

def load_abalone_data(data_path, attr_path):
    with open(attr_path, 'r', encoding='utf-8') as f:
        columns = [line.strip() for line in f if line.strip()]

    df = pd.read_csv(data_path, header=None, names=columns)
    return df

def label_encode(label_series):
    encoder = LabelEncoder()
    encoded = encoder.fit_transform(label_series)
    print('Label Encoding 결과:', encoded[:10])
    print('클래스:', list(encoder.classes_))
    return encoded, encoder

def one_hot_encode(encoded_labels):
    encoder = OneHotEncoder(sparse_output=False)
    encoded_2d = np.array(encoded_labels).reshape(-1, 1)
    one_hot = encoder.fit_transform(encoded_2d)
    print('\nOne-Hot Encoding 결과 (앞 5개):\n', one_hot[:5])
    print('카테고리:', encoder.categories_)
    return one_hot

def handle_missing_values(df):
    print('\n[보너스 과제] 결측값 처리 예시')
    df_with_nan = df.copy()
    df_with_nan.iloc[0, 1] = np.nan  # 임의 결측치 삽입
    print('결측값 전:\n', df_with_nan.head())
    df_filled = df_with_nan.fillna(df_with_nan.mean(numeric_only=True))
    print('결측값 처리 후:\n', df_filled.head())

def handle_noise(df):
    print('\n[보너스 과제] 노이즈 처리 예시')
    df_noisy = df.copy()
    df_noisy.iloc[0, 2] = 9999  # 임의 노이즈 삽입
    print('노이즈 전:\n', df_noisy.head())
    # 단순 평균 대체
    col_mean = df_noisy[df_noisy.iloc[:, 2] < 1000].iloc[:, 2].mean()
    df_noisy.iloc[0, 2] = col_mean
    print('노이즈 처리 후:\n', df_noisy.head())
```

```python
def handle_outliers(df):
    print('\n[보너스 과제] 이상값 처리 예시')
    df_outlier = df.copy()
    col = df_outlier.iloc[:, 3]
    q1 = col.quantile(0.25)
    q3 = col.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    outliers = (col < lower_bound) | (col > upper_bound)
    print('이상값 개수:', outliers.sum())
    df_outlier.loc[outliers, col.name] = col.median()
    print('이상값 처리 후 일부:\n', df_outlier[outliers].head())


def main():
    data_path = 'src/Process9/9-4/9-4-abalone.txt'
    attr_path = 'src/Process9/9-4/9-4-abalone_attributes.txt'

    df = load_abalone_data(data_path, attr_path)

    label = df['Sex']
    data = df.drop(columns=['Sex'])

    # Label Encoding
    encoded_label, label_encoder = label_encode(label)

    # One-Hot Encoding
    one_hot_encoded = one_hot_encode(encoded_label)

    # [보너스 과제] 데이터 전처리 예시
    handle_missing_values(data)
    handle_noise(data)
    handle_outliers(data)


if __name__ == '__main__':
    main()
```

문제 6. 분명하게 말해 두기 - 구현화면



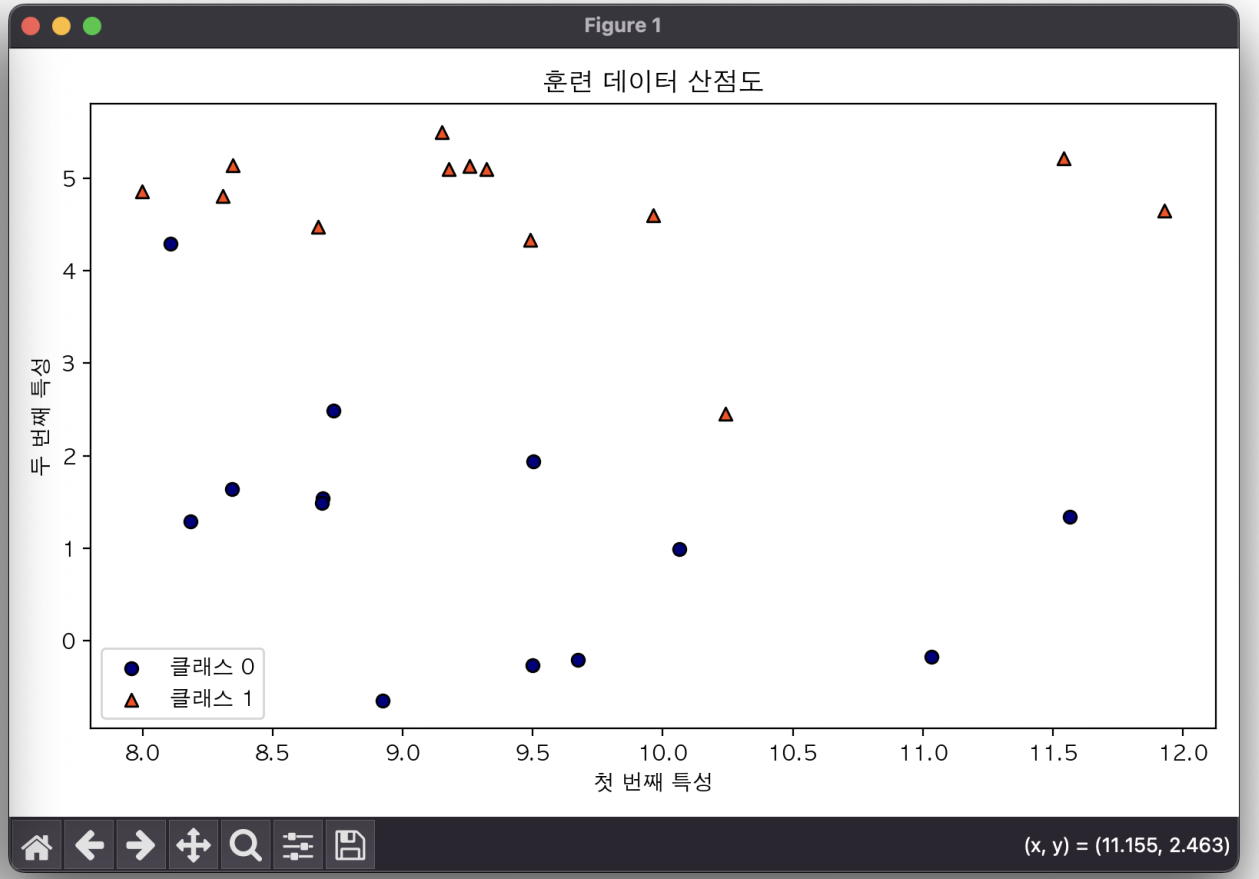
문제 7. 맛있는 토마토 찾기 - 코드

```python
# 과정 9 – (문제7) 맛있는 토마토 찾기
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import mglearn
import matplotlib

matplotlib.rc('font', family='AppleGothic')
plt.rcParams['axes.unicode_minus'] = False

def plot_scatter(x, y, title='산점도 시각화'):
    plt.figure(figsize=(8, 5))
    for label, marker, color in zip([0, 1], ['o', '^'], ['navy', 'orangered']):
        plt.scatter(
            x[y == label, 0], x[y == label, 1],
            marker=marker, color=color, label=f'클래스 {label}', edgecolor='black'
        )
    plt.xlabel('첫 번째 특성')
    plt.ylabel('두 번째 특성')
    plt.legend()
    plt.title(title)
    plt.tight_layout()
    plt.show()
```

```python
def main():
    # 데이터 생성
    x, y = mglearn.datasets.make_forge()

    # 산점도 시각화
    plot_scatter(x, y, '훈련 데이터 산점도')

    # 데이터 분할
    x_train, x_test, y_train, y_test = train_test_split(
        x, y, random_state=0
    )

    # KNN 학습 및 평가 (n_neighbors=3)
    model = KNeighborsClassifier(n_neighbors=3)
    model.fit(x_train, y_train)
    accuracy = model.score(x_test, y_test)
    print('기본 모델 정확도 (n_neighbors=3):', accuracy)

    # [보너스 과제] n_neighbors = 1 ~ 9 변화에 따른 정확도
    print('\n[보너스 과제] n_neighbors 변화에 따른 테스트 정확도:')
    for k in range(1, 10):
        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(x_train, y_train)
        acc = knn.score(x_test, y_test)
        print(f'n_neighbors = {k} → 정확도: {acc:.2f}')


if __name__ == '__main__':
    main()
```

문제 7. 맛있는 토마토 찾기 - 구현화면

```
swpilotPyhon5 — -zsh — 99×33
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process9/9-7/main.py
기본 모델 정확도 (n_neighbors=3): 0.8571428571428571

[보너스 과제] n_neighbors 변화에 따른 테스트 정확도:
n_neighbors = 1 → 정확도: 0.86
n_neighbors = 2 → 정확도: 0.86
n_neighbors = 3 → 정확도: 0.86
n_neighbors = 4 → 정확도: 0.86
n_neighbors = 5 → 정확도: 0.86
n_neighbors = 6 → 정확도: 0.86
n_neighbors = 7 → 정확도: 0.86
n_neighbors = 8 → 정확도: 0.86
n_neighbors = 9 → 정확도: 0.86
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```



문제 8. 진짜 맛있는 토마토 찾기 - 코드

```python
# 과정 9 – (문제8) 진짜 맛있는 토마토 찾기
import matplotlib.pyplot as plt
from mglearn.datasets import make_forge
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import mglearn
import numpy as np
import matplotlib

matplotlib.rc('font', family='AppleGothic')
plt.rcParams['axes.unicode_minus'] = False

def plot_decision_boundary(model, x, y, title):
    mglearn.plots.plot_2d_separator(model, x, fill=True, alpha=0.4)
    mglearn.discrete_scatter(x[:, 0], x[:, 1], y)
    plt.title(title)
    plt.xlabel('첫 번째 특성')
    plt.ylabel('두 번째 특성')
    plt.tight_layout()
    plt.show()

def main():
    # 데이터 생성 및 분리
    x, y = mglearn.datasets.make_forge()
    x_train, x_test, y_train, y_test = train_test_split(
        x, y, random_state=0
    )

    training_accuracy = []
    test_accuracy = []
    neighbor_settings = range(1, 11)
```

```python
    for n_neighbors in neighbor_settings:
        model = KNeighborsClassifier(n_neighbors=n_neighbors)
        model.fit(x_train, y_train)
        train_acc = model.score(x_train, y_train)
        test_acc = model.score(x_test, y_test)
        training_accuracy.append(train_acc)
        test_accuracy.append(test_acc)

    # 꺾은선 그래프 출력
    plt.figure(figsize=(8, 5))
    plt.plot(neighbor_settings, training_accuracy, label='훈련 정확도')
    plt.plot(neighbor_settings, test_accuracy, label='테스트 정확도')
    plt.xlabel('n_neighbors')
    plt.ylabel('정확도 (accuracy)')
    plt.title('KNN 정확도 비교')
    plt.legend()
    plt.ylim(0.5, 1)
    plt.tight_layout()
    plt.grid(False)
    plt.show()

    # [보너스 과제] 결정 경계 시각화 (n = 1, 3, 9)
    for k in [1, 3, 9]:
        model = KNeighborsClassifier(n_neighbors=k)
        model.fit(x, y)
        plot_decision_boundary(model, x, y, f'결정 경계 (n_neighbors = {k})')

if __name__ == '__main__':
    main()
```

문제 8. 진짜 맛있는 토마토 찾기 - 구현화면